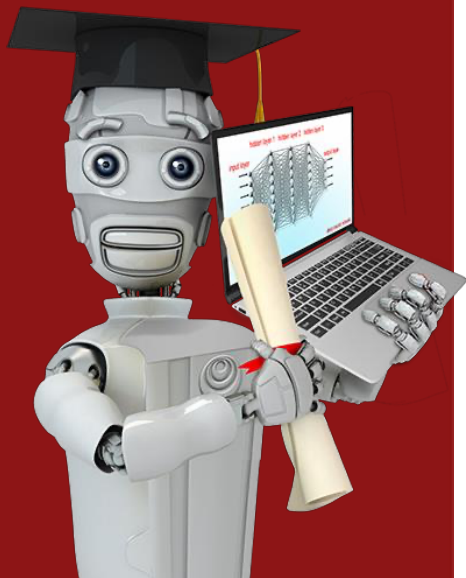




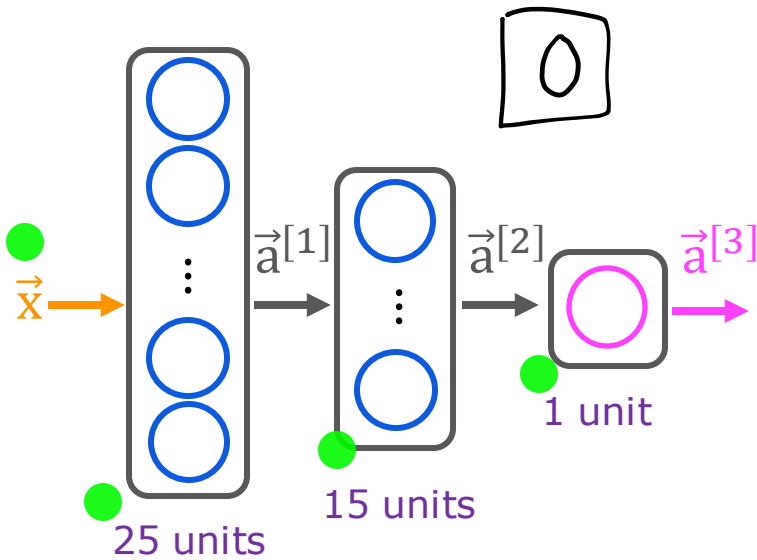
Stanford
ONLINE



Neural Network Training

TensorFlow implementation

Train a Neural Network in TensorFlow



Given set of (x, y) examples

How to build and train this in code?

```
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense

model = Sequential([
    Dense(units=25, activation='sigmoid')
    Dense(units=15, activation='sigmoid')
    Dense(units=1, activation='sigmoid')
])

from tensorflow.keras.losses import BinaryCrossentropy

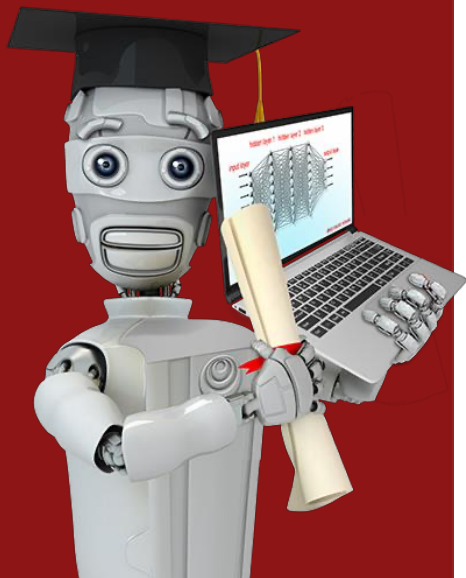
model.compile(loss=BinaryCrossentropy())

model.fit(X, Y, epochs=100)
```

epochs: number of steps in gradient descent



Stanford
ONLINE



Neural Network Training

Training Details

Model Training Steps TensorFlow

①

specify how to compute output given input x and parameters w, b (define model)

$$f_{\vec{w}, b}(\vec{x}) = ?$$

②

specify loss and cost

$$L(f_{\vec{w}, b}(\vec{x}), y) \quad \text{1 example}$$

$$J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^m L(f_{\vec{w}, b}(\vec{x}^{(i)}), y^{(i)})$$

③

Train on data to minimize $J(\vec{w}, b)$

logistic regression

```
z = np.dot(w, x) + b  
f_x = 1 / (1 + np.exp(-z))
```



logistic loss

```
loss = -y * np.log(f_x)  
       -(1-y) * np.log(1-f_x)
```

```
w = w - alpha * dj_dw  
b = b - alpha * dj_db
```

neural network

```
model = Sequential([  
    Dense(...)  
    Dense(...)  
    Dense(...)])
```

binary cross entropy

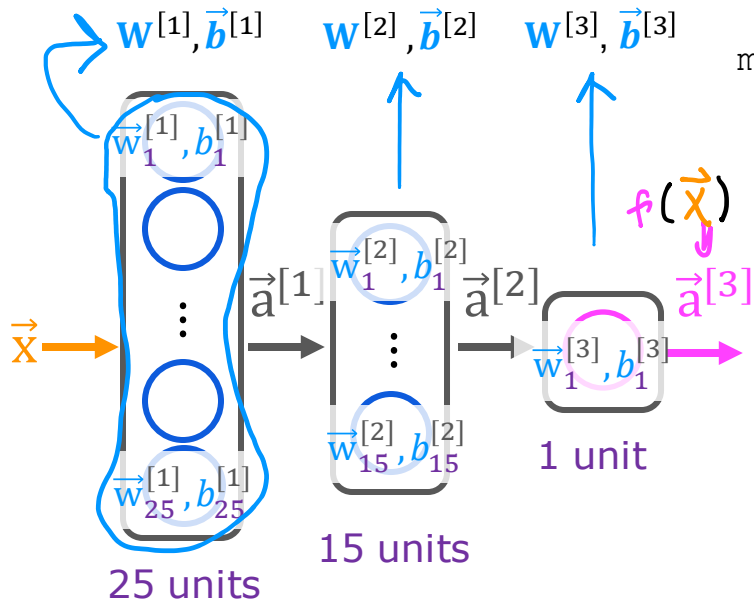
```
model.compile(  
    loss=BinaryCrossentropy())
```

```
model.fit(X, y, epochs=100)
```

1. Create the model

define the model

$$f(\vec{x}) = ?$$



```
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
```

```
model = Sequential([
    Dense(units=25, activation='sigmoid')
    Dense(units=15, activation='sigmoid')
    Dense(units=1, activation='sigmoid')
])
```

2. Loss and cost functions

Mnist digit
classification problem

binary classification

$$L(f(\vec{x}), y) = -y \log(f(\vec{x})) - (1 - y) \log(1 - f(\vec{x}))$$

compare prediction vs. target

logistic loss

also known as binary cross entropy

```
model.compile(loss= BinaryCrossentropy())
```

regression

(predicting numbers
and not categories)

mean squared error

```
model.compile(loss= MeanSquaredError())
```

$$J(\mathbf{W}, \mathbf{B}) = \frac{1}{m} \sum_{i=1}^m L(f(\vec{x}^{(i)}), y^{(i)})$$

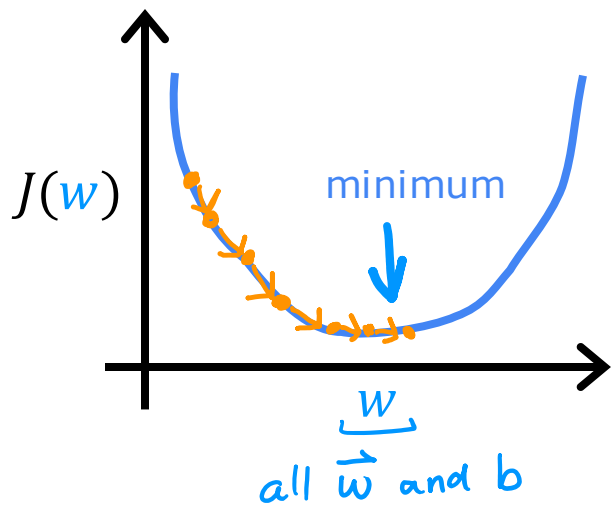
$\mathbf{w}^{[1]}, \mathbf{w}^{[2]}, \mathbf{w}^{[3]}$ $\vec{b}^{[1]}, \vec{b}^{[2]}, \vec{b}^{[3]}$ $f_{\mathbf{W}, \mathbf{B}}(\vec{x})$

```
from tensorflow.keras.losses import  
BinaryCrossentropy
```



```
from tensorflow.keras.losses import  
MeanSquaredError
```

3. Gradient descent



repeat {

$$w_j^{[l]} = w_j^{[l]} - \alpha \frac{\partial}{\partial w_j} J(\vec{w}, b)$$

$$b_j^{[l]} = b_j^{[l]} - \alpha \frac{\partial}{\partial b} J(\vec{w}, b)$$

} Compute derivatives
for gradient descent
using "backpropagation"

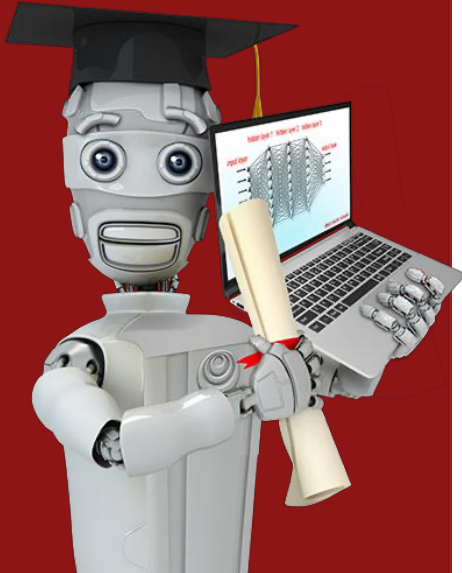
```
model.fit(X, y, epochs=100)
```

Neural network libraries

Use code libraries instead of coding "from scratch"



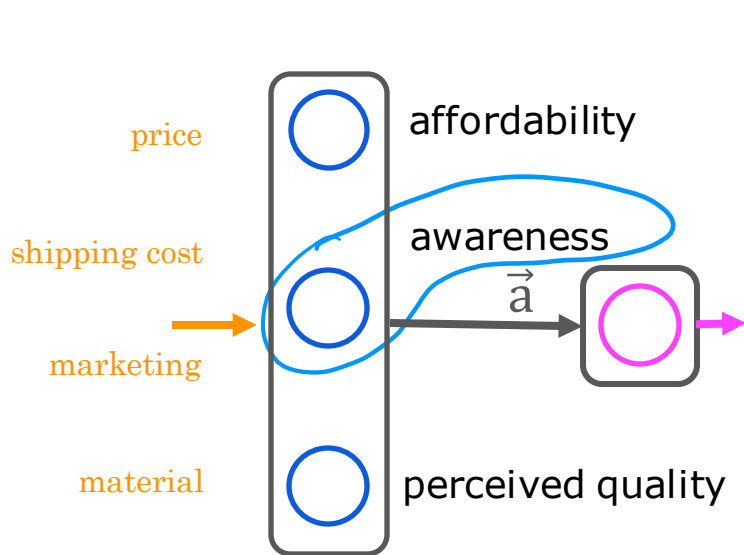
Good to understand the implementation
(for tuning and debugging).



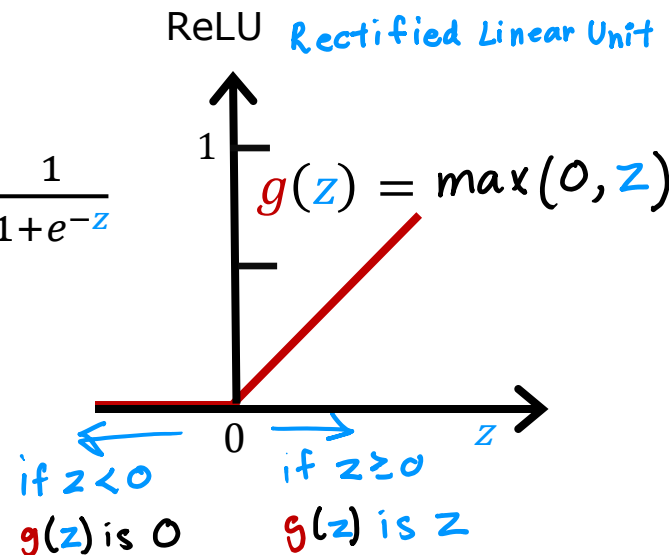
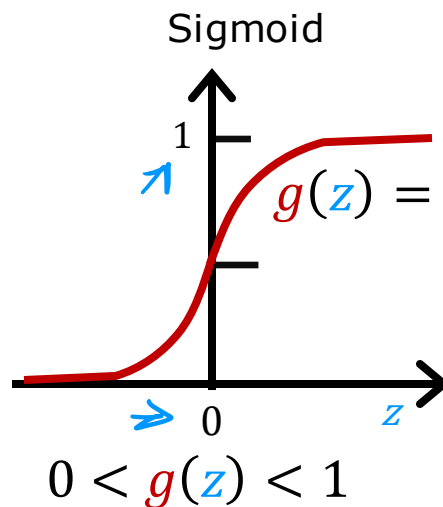
Activation Functions

Alternatives to the
sigmoid activation

Demand Prediction Example



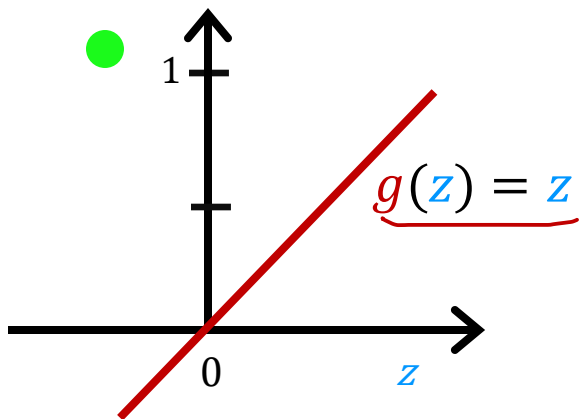
$$a_2^{[1]} = g(\overbrace{\vec{w}_2^{[1]} \cdot \vec{x} + b_2^{[1]}}^z)$$



Examples of Activation Functions

"No activation function"

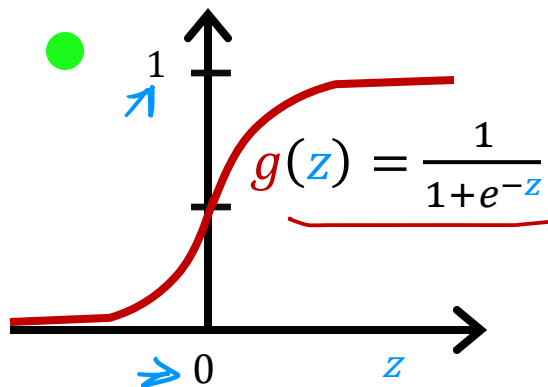
Linear activation function



$$a = g(z) = \underbrace{\vec{w} \cdot \vec{x} + b}_z$$

$$a_2^{[1]} = g(\overbrace{\vec{w}_2^{[1]} \cdot \vec{x} + b_2^{[1]}}^z)$$

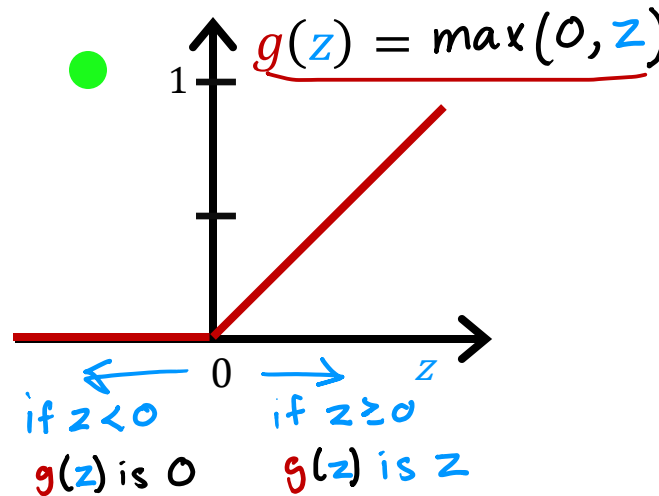
Sigmoid

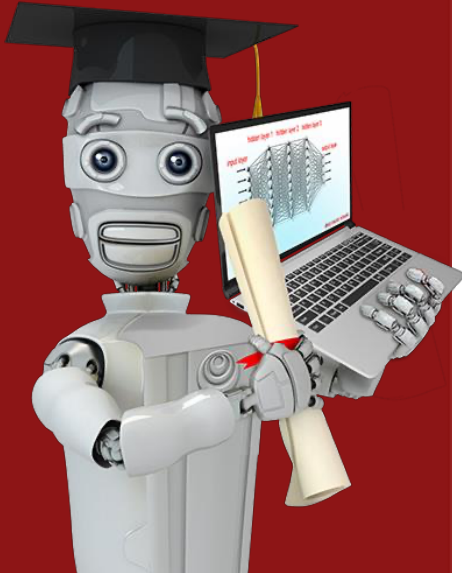


$$0 < g(z) < 1$$

Later: softmax activation

ReLU Rectified Linear Unit

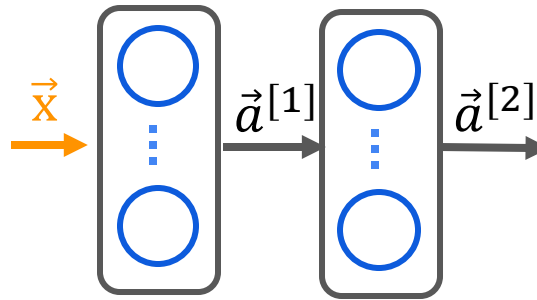




Activation Functions

Choosing activation functions

Output Layer

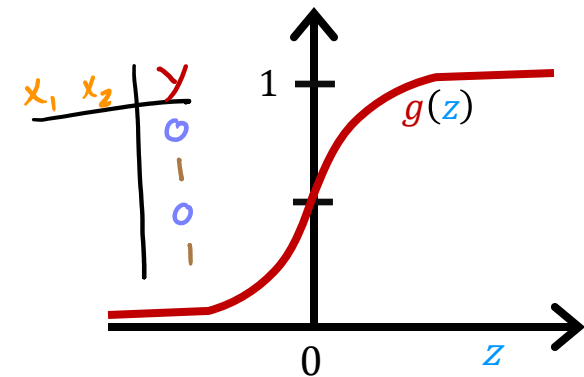


$$\vec{a}^{[3]} = f(\vec{x})$$

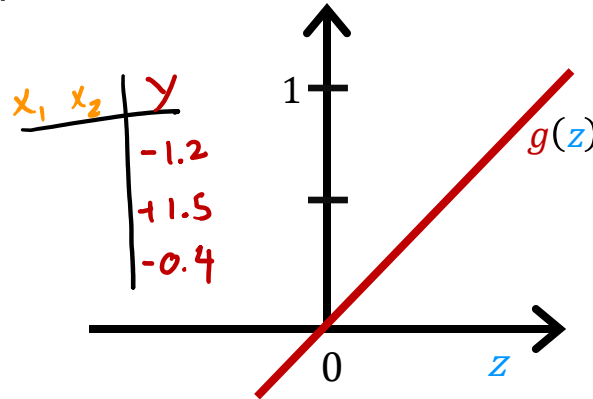
$$f(\vec{x}) = a_1^{[3]} = g(z_1^{[3]})$$

Choosing $g(z)$ for output layer?

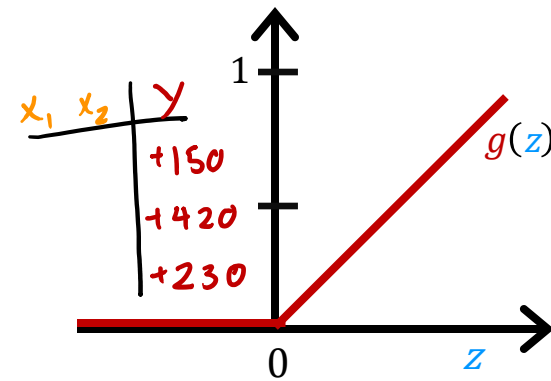
- Binary classification
- Sigmoid
- $y = 0/1$



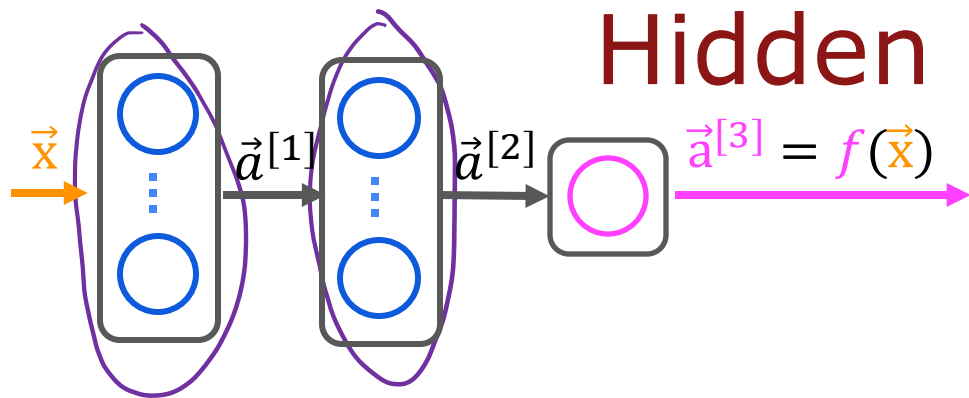
- Regression
- Linear activation function
- $y = +/-$



- Regression
- ReLU
- $Y = 0$ or $+$

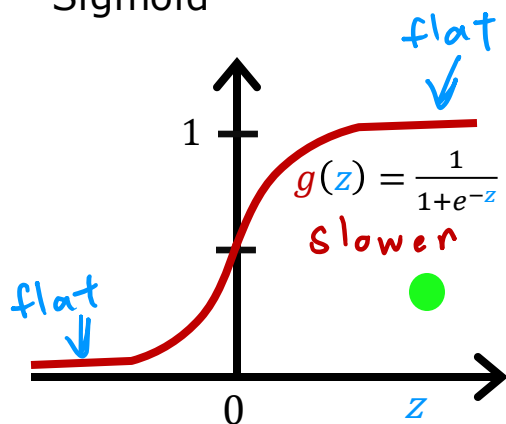


Hidden Layer

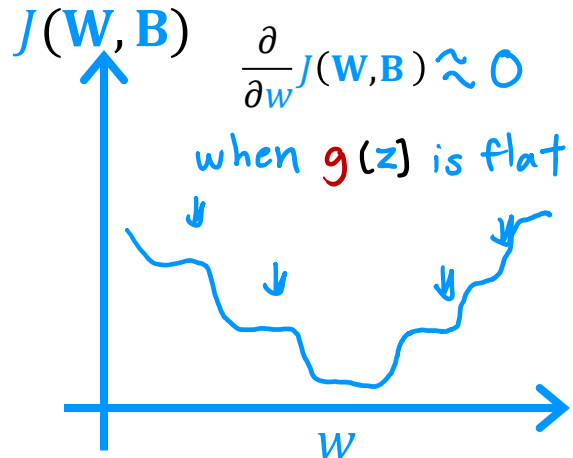


Choosing $g(z)$ for hidden layer

Sigmoid



$J(W, B)$

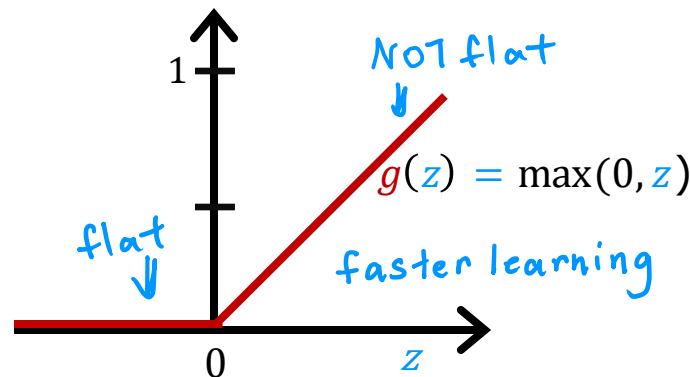


$$\frac{\partial}{\partial W} J(W, B) \approx 0$$

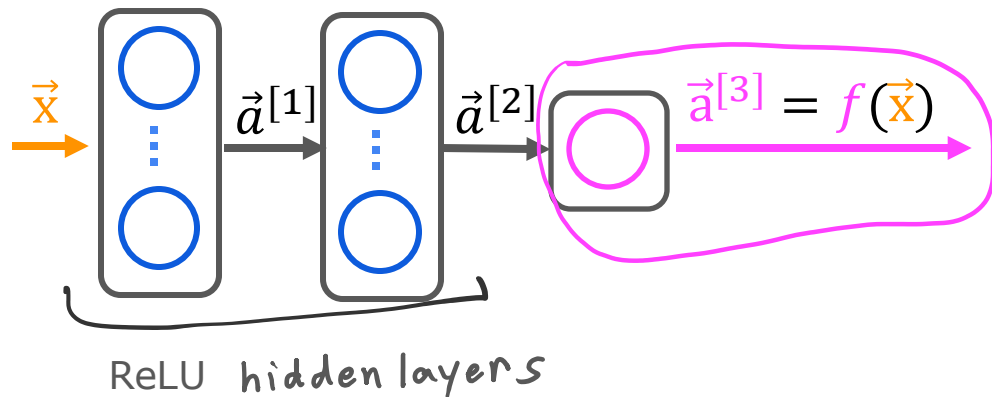
most common choice

ReLU

faster



Choosing Activation Summary



binary classification

activation='sigmoid'

regression y negative / positive

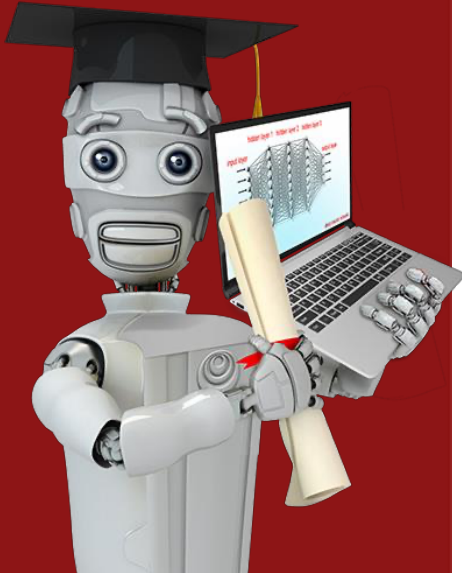
activation='linear'

regression $y \geq 0$

activation='relu'

```
from tf.keras.layers import Dense
model = Sequential([
    Dense(units=25, activation='relu'),    layer1
    Dense(units=15, activation='relu'),    layer2
    Dense(units=1, activation='sigmoid')   layer3
])
```

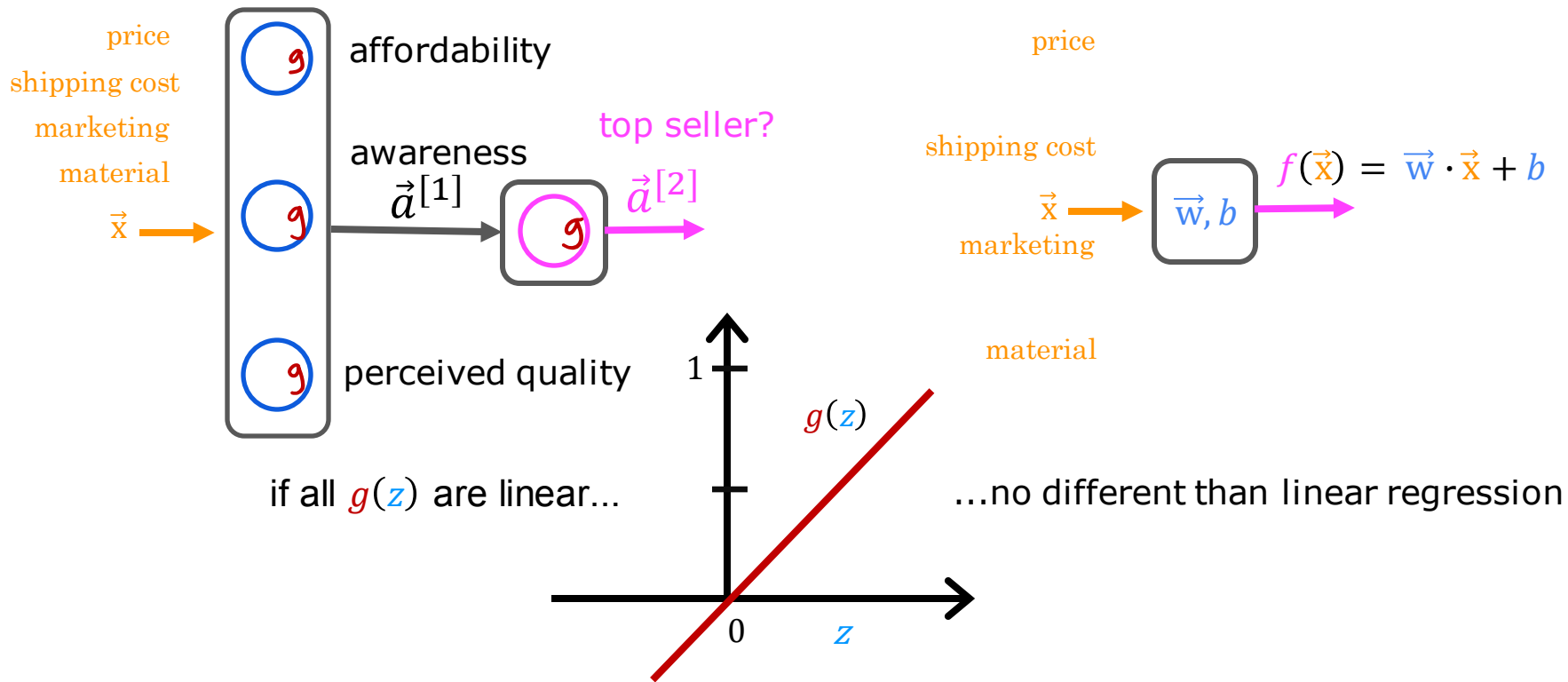
or 'linear'
or 'relu'



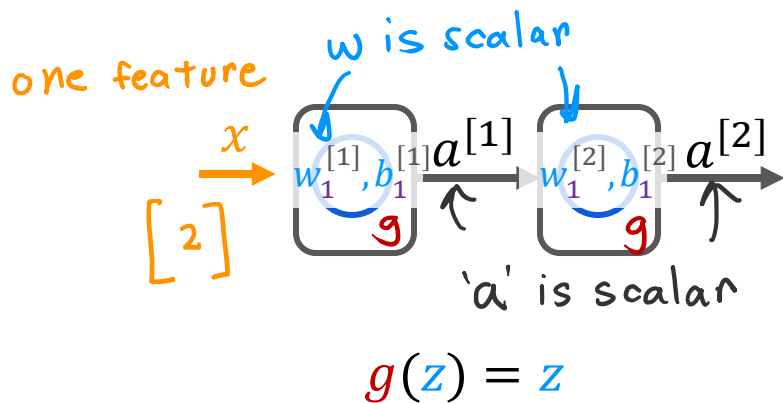
Activation Functions

Why do we need
activation functions?

Why do we need activation functions?



Linear Example



$$a^{[1]} = w_1^{[1]} x + b_1^{[1]}$$

$$a^{[2]} = w_1^{[2]} a^{[1]} + b_1^{[2]}$$

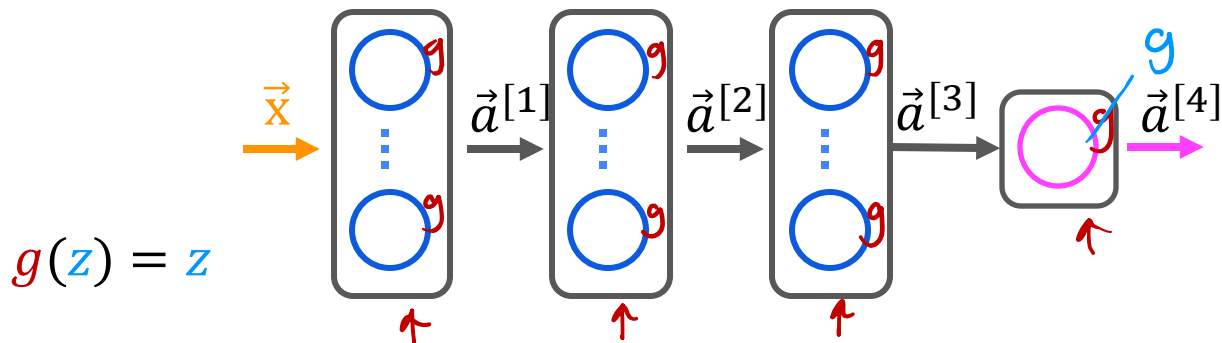
$$= w_1^{[2]} (w_1^{[1]} x + b_1^{[1]}) + b_1^{[2]}$$

$$\vec{a}^{[2]} = (\underbrace{\vec{w}_1^{[2]} \vec{w}_1^{[1]}}_w) x + \underbrace{w_1^{[2]} b_1^{[1]} + b_1^{[2]}}_b$$

$$\vec{a}^{[2]} = w x + b$$

$$f(x) = wx + b \quad \text{linear regression}$$

Example



$$g(z) = z$$

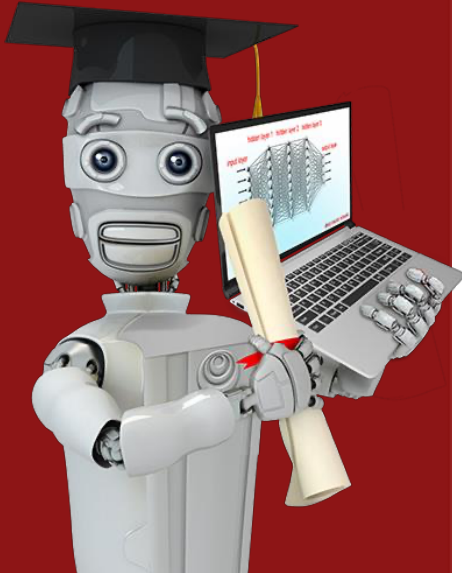
$$\vec{a}^{[4]} = \vec{w}_1^{[4]} \cdot \vec{a}^{[3]} + b_1^{[4]}$$

all linear (including output)
↳ equivalent to linear regression

$$\vec{a}^{[4]} = \frac{1}{1 + e^{-(\vec{w}_1^{[4]} \cdot \vec{a}^{[3]} + b_1^{[4]})}}$$

output activation is sigmoid
(hidden layers still linear)
↳ equivalent to logistic regression

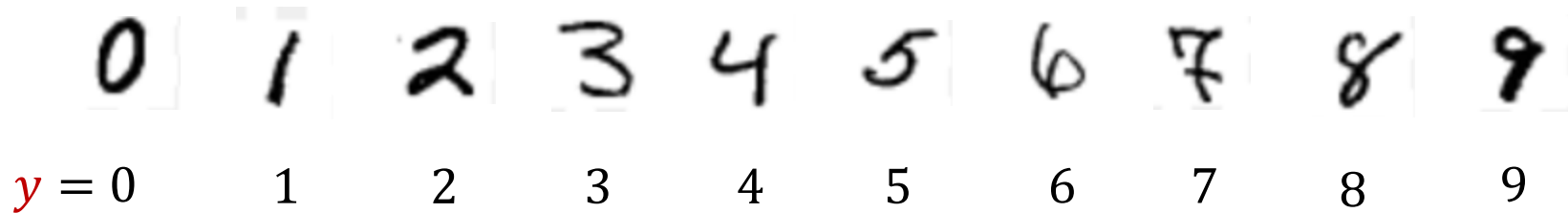
Don't use linear activations in hidden layers (use ReLU)



Multiclass Classification

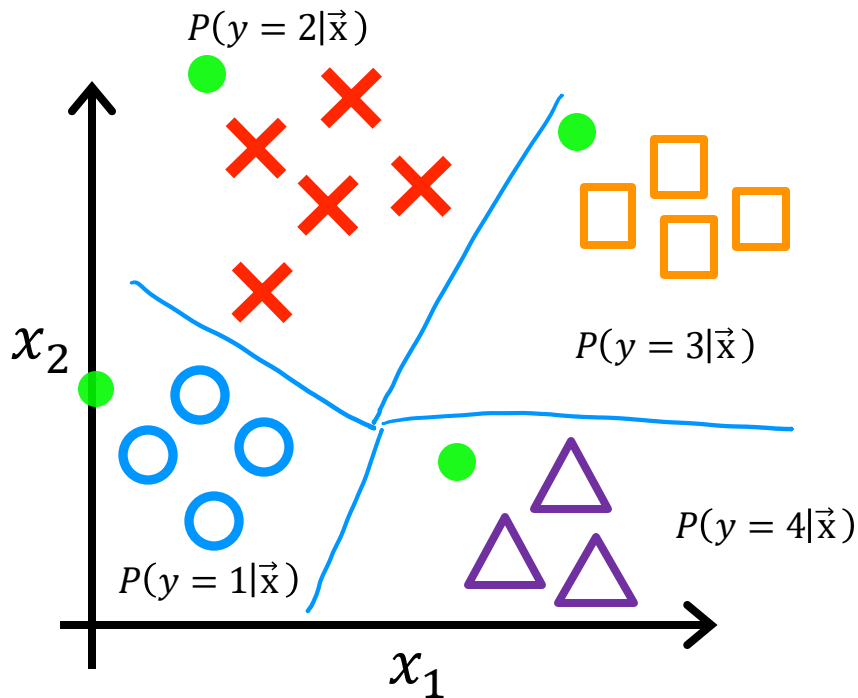
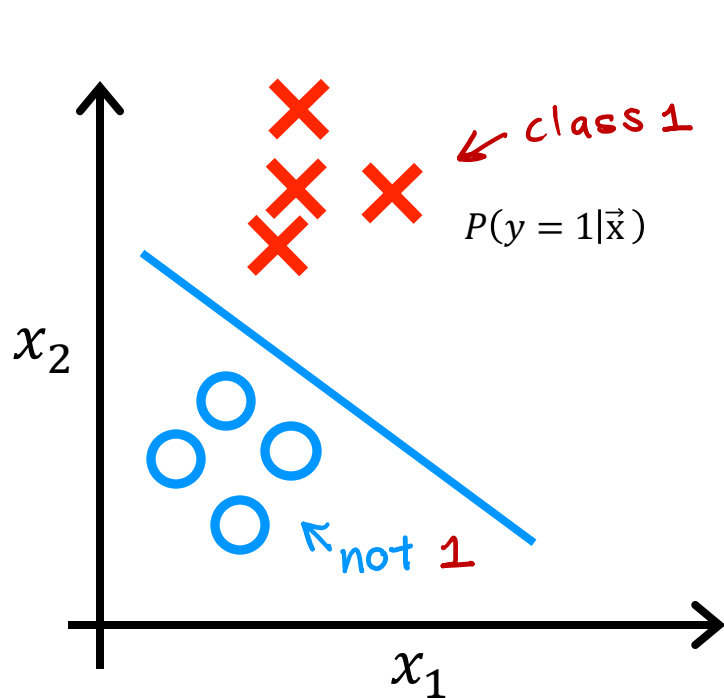
Multiclass

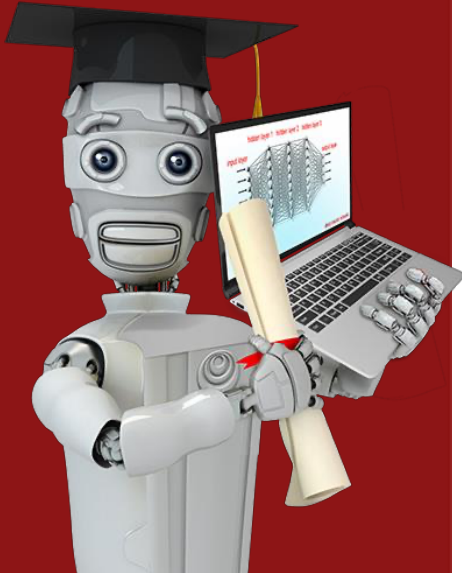
MNIST example



multiclass classification problem:
target y can take on more than two possible values

Multiclass classification example





Multiclass Classification

Softmax

Logistic regression
(2 possible output values)

$$z = \vec{w} \cdot \vec{x} + b$$

$$\times a_1 = g(z) = \frac{1}{1+e^{-z}} = P(y=1|\vec{x}) \quad 0.71$$

$$\circ a_2 = 1 - a_1 = P(y=0|\vec{x}) \quad 0.29$$

Softmax regression
(N possible outputs) $y=1,2,3,\dots,N$

$$z_j = \vec{w}_j \cdot \vec{x} + b_j \quad j = 1, \dots, N$$

parameters $w_1, w_2, \dots, w_N$
 $b_1, b_2, \dots, b_N$

$$a_j = \frac{e^{z_j}}{\sum_{k=1}^N e^{z_k}} = P(y=j|\vec{x})$$

$$\text{note: } a_1 + a_2 + \dots + a_N = 1$$

Softmax regression (4 possible outputs) $y=1,2,3,4$

$$\times z_1 = \vec{w}_1 \cdot \vec{x} + b_1$$

$$a_1 = \frac{e^{z_1}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}} \\ = P(y=1|\vec{x}) \quad 0.30$$

$$\circ z_2 = \vec{w}_2 \cdot \vec{x} + b_2$$

$$a_2 = \frac{e^{z_2}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}} \\ = P(y=2|\vec{x}) \quad 0.20$$

$$\square z_3 = \vec{w}_3 \cdot \vec{x} + b_3$$

$$a_3 = \frac{e^{z_3}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}} \\ = P(y=3|\vec{x}) \quad 0.15$$

$$\triangle z_4 = \vec{w}_4 \cdot \vec{x} + b_4$$

$$a_4 = \frac{e^{z_4}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}} \\ = P(y=4|\vec{x}) \quad 0.35$$

Cost

Logistic regression

$$z = \vec{w} \cdot \vec{x} + b$$

$$a_1 = g(z) = \frac{1}{1 + e^{-z}} = P(y = 1|\vec{x})$$

$$a_2 = 1 - a_1 = P(y = 0|\vec{x})$$

$$\text{loss} = \underbrace{-y \log a_1}_{\text{if } y=1} - \underbrace{(1-y) \log(1-a_1)}_{\text{if } y=0}$$

$$J(\vec{w}, b) = \text{average loss}$$

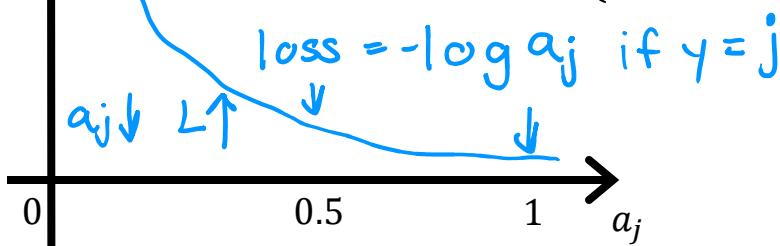
Softmax regression

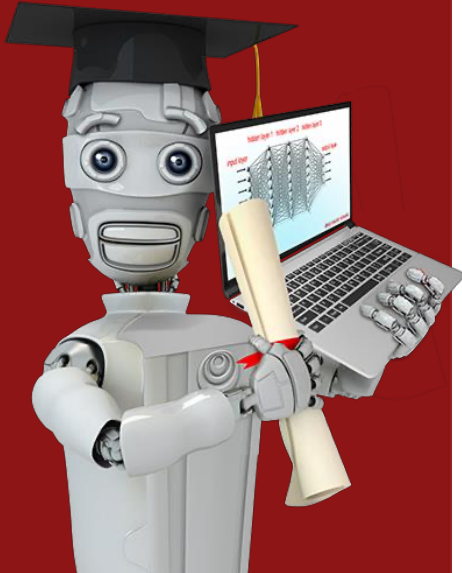
$$a_1 = \frac{e^{z_1}}{e^{z_1} + e^{z_2} + \dots + e^{z_N}} = P(y = 1|\vec{x})$$

$$\vdots$$
$$a_N = \frac{e^{z_N}}{e^{z_1} + e^{z_2} + \dots + e^{z_N}} = P(y = N|\vec{x})$$

Crossentropy loss

$$\text{loss}(a_1, \dots, a_N, y) = \begin{cases} -\log a_1 & \text{if } y = 1 \\ -\log a_2 & \text{if } y = 2 \\ \vdots \\ -\log a_N & \text{if } y = N \end{cases}$$

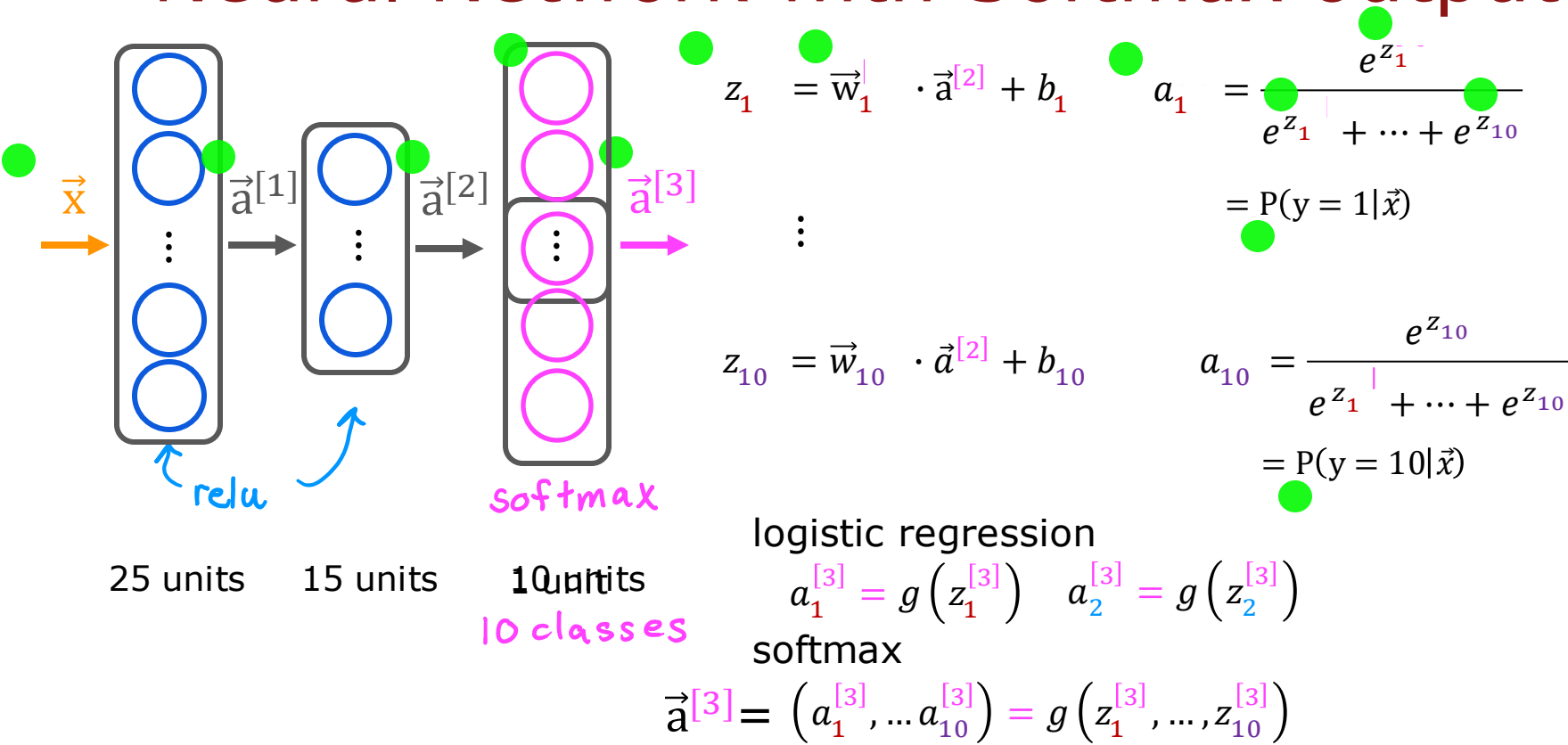




Multiclass Classification

Neural Network with
Softmax output

Neural Network with Softmax output



MNIST with softmax

① specify the model

$$f_{\vec{w},b}(\vec{x}) = ?$$

```
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense

model = Sequential([
    • Dense(units=25, activation='relu')
    • Dense(units=15, activation='relu')
    • Dense(units=10, activation='softmax')
])
```

② specify loss and cost

$$L(f_{\vec{w},b}(\vec{x}), y)$$

```
from tensorflow.keras.losses import
    SparseCategoricalCrossentropy

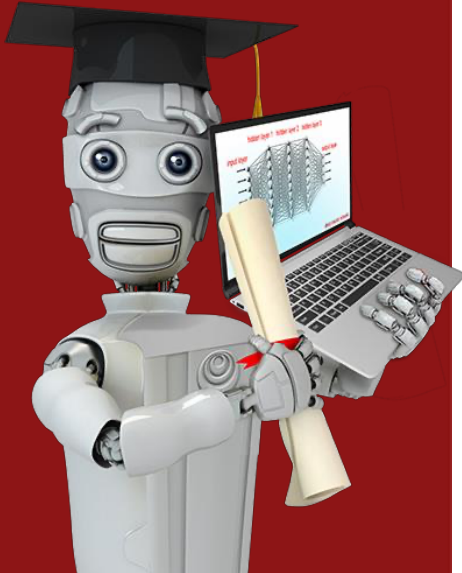
model.compile(loss= SparseCategoricalCrossentropy() )

model.fit(X,Y,epochs=100)
```

③ Train on data to minimize $J(\vec{w},b)$

Note: better (recommended) version later.

Don't use the version shown here!



Multiclass Classification

Improved implementation
of softmax

Numerical Roundoff Errors

option 1

$$x = \frac{2}{10,000}$$

option 2

$$x = \left(1 + \frac{1}{10,000}\right) - \left(1 - \frac{1}{10,000}\right) =$$


Numerical Roundoff Errors

More numerically accurate implementation of logistic loss:

$$1 + \frac{1}{10,000} \quad 1 - \frac{1}{10,000}$$

Logistic regression:

$$\boxed{a} = g(z) = \frac{1}{1 + e^{-z}}$$

```
model = Sequential([  
    Dense(units=25, activation='relu')  
    Dense(units=15, activation='relu') 'linear'  
    Dense(units=10, activation='sigmoid')
```

Original loss

$$\text{loss} = -y \log(\boxed{a}) - (1 - y) \log(1 - \boxed{a})$$

```
model.compile(loss=BinaryCrossEntropy() )  
model.compile(loss=BinaryCrossEntropy(from_logits=True) )
```

More accurate loss (in code)

$$\text{loss} = -y \log\left(\frac{1}{1 + e^{-z}}\right) - (1 - y) \log\left(1 - \frac{1}{1 + e^{-z}}\right)$$

logit: z

More numerically accurate implementation of softmax

Softmax regression

$$(a_1, \dots, a_{10}) = g(z_1, \dots, z_{10})$$

$$\text{Loss} = L(\vec{a}, y) = \begin{cases} -\log a_1 & \text{if } y = 1 \\ \vdots \\ -\log a_{10} & \text{if } y = 10 \end{cases}$$

```
model = Sequential([  
    Dense(units=25, activation='relu')  
    Dense(units=15, activation='relu')  
    Dense(units=10, activation='softmax')
```

'linear'

```
model.compile(loss=SparseCategoricalCrossEntropy())
```

More Accurate

$$L(\vec{a}, y) = \begin{cases} -\log \frac{e^{z_1}}{e^{z_1} + \dots + e^{z_{10}}} & \text{if } y = 1 \\ \vdots \\ -\log \frac{e^{z_{10}}}{e^{z_1} + \dots + e^{z_{10}}} & \text{if } y = 10 \end{cases}$$

```
model.compile(loss=SparseCrossEntropy(from_logits=True))
```


MNIST (more numerically accurate)

```
model    import tensorflow as tf
         from tensorflow.keras import Sequential
         from tensorflow.keras.layers import Dense
         model = Sequential([
             Dense(units=25, activation='relu')
             Dense(units=15, activation='relu')
             Dense(units=10, activation='linear') ])

loss     from tensorflow.keras.losses import
         SparseCategoricalCrossentropy

         model.compile(..., loss=SparseCategoricalCrossentropy(from_logits=True) )

fit      model.fit(X,Y,epochs=100)

predict  logits = model(X) ← not  $a_1 \dots a_{10}$ 
         f_x = tf.nn.softmax(logits)           is  $z_1 \dots z_{10}$ 
```

logistic regression (more numerically accurate)

```
model model = Sequential([
    Dense(units=25, activation='sigmoid')
    Dense(units=15, activation='sigmoid')
    Dense(units=1, activation='linear')
from tensorflow.keras.losses import
    BinaryCrossentropy

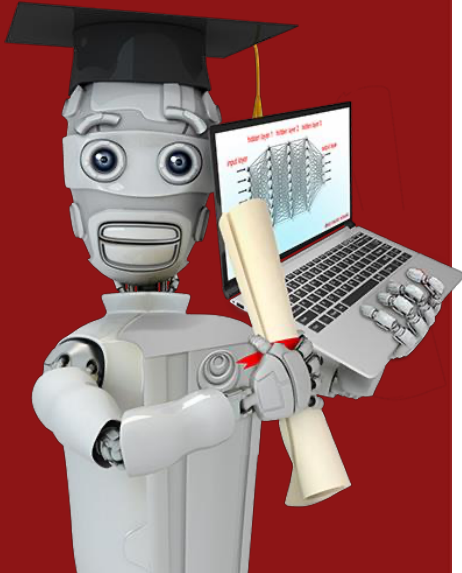
loss model.compile(..., BinaryCrossentropy(from_logits=True))

model.fit(X,Y,epochs=100)

fit logit = model(X)

predict f_x = tf.nn.sigmoid(logit)
```

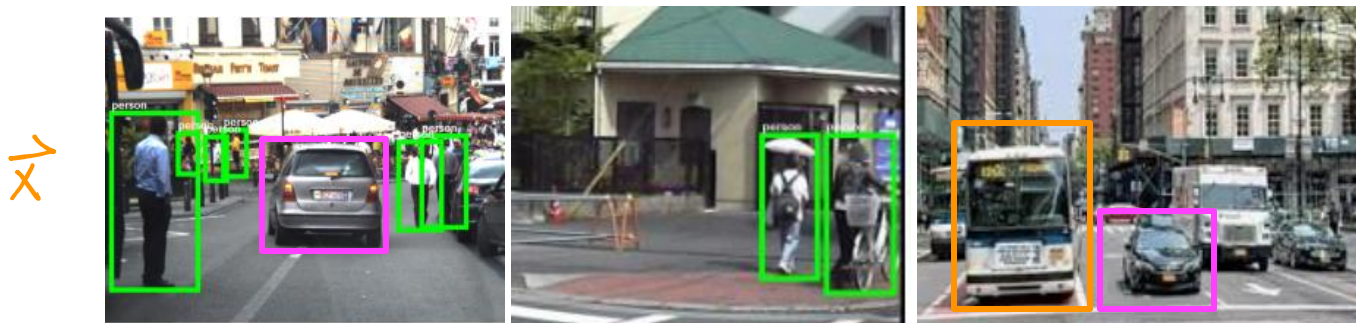
Handwritten annotations:
A blue bracket underlines 'linear' in the model definition.
A blue bracket underlines 'from_logits=True' in the compile step.
A blue arrow points from the variable 'logit' in the fit step to the 'logit' argument in the sigmoid function in the predict step.
A blue bracket underlines the entire 'tf.nn.sigmoid(logit)' expression in the predict step.



Multi-label Classification

Classification with
multiple outputs
(Optional)

Multi-label Classification



Is there a car?

yes
no
yes $y = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}$

Is there a bus?

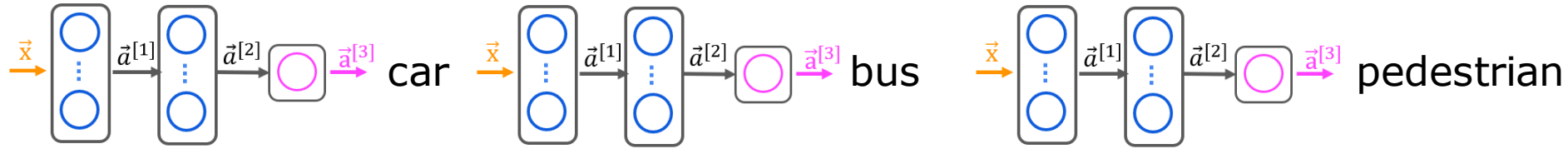
no
no
yes $y = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$

Is there a pedestrian

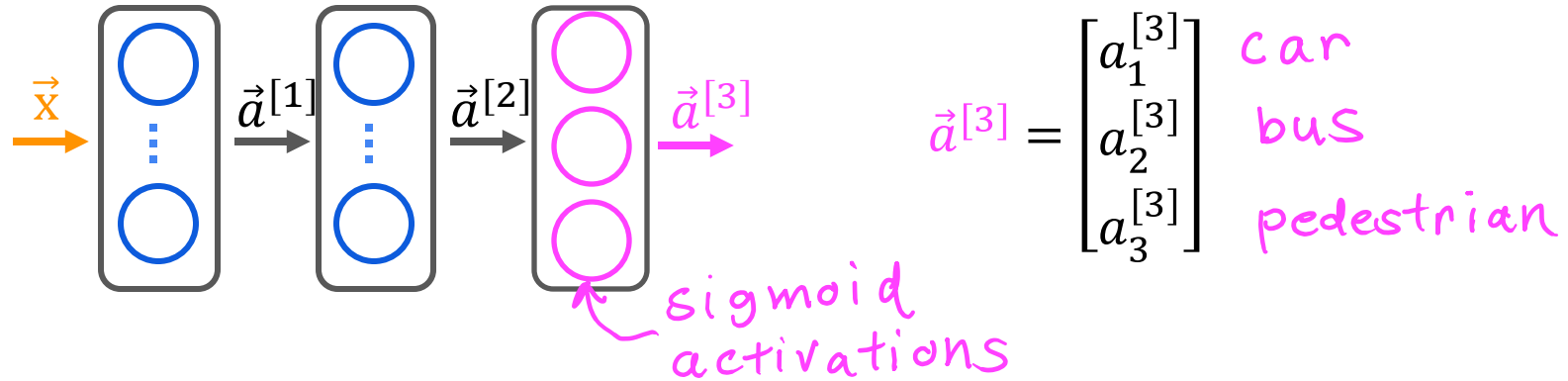
yes
yes
no $y = \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}$



Multiple classes

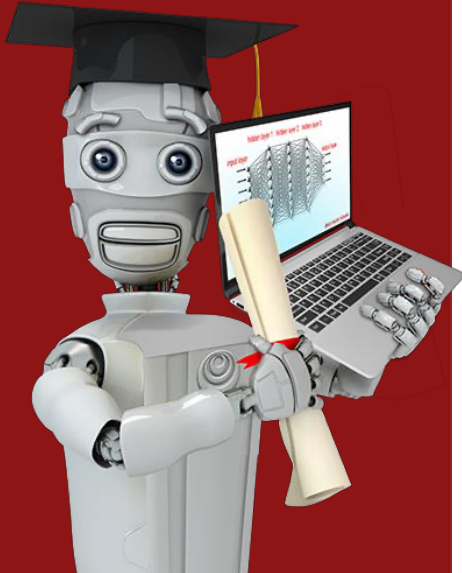


Alternatively, train one neural network with three outputs





Stanford
ONLINE



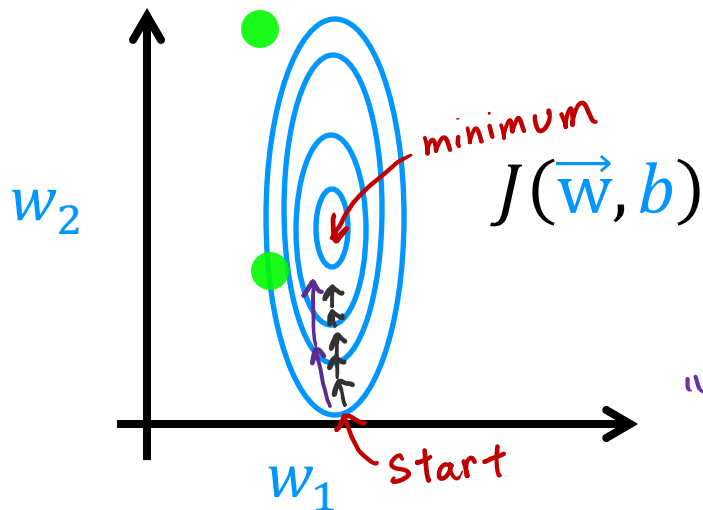
Additional Neural Network Concepts

Advanced Optimization

Gradient Descent

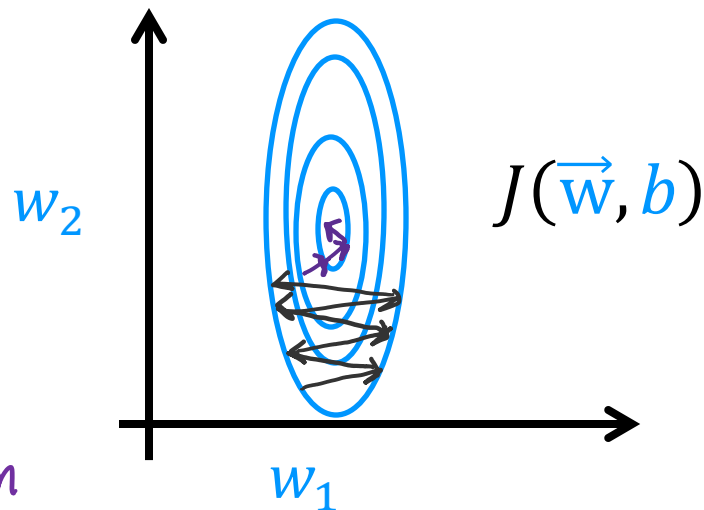
$$w_j = w_j - \alpha \frac{\partial}{\partial w_j} J(\vec{w}, b)$$

α learning rate



Go faster – increase α

"Adam" algorithm



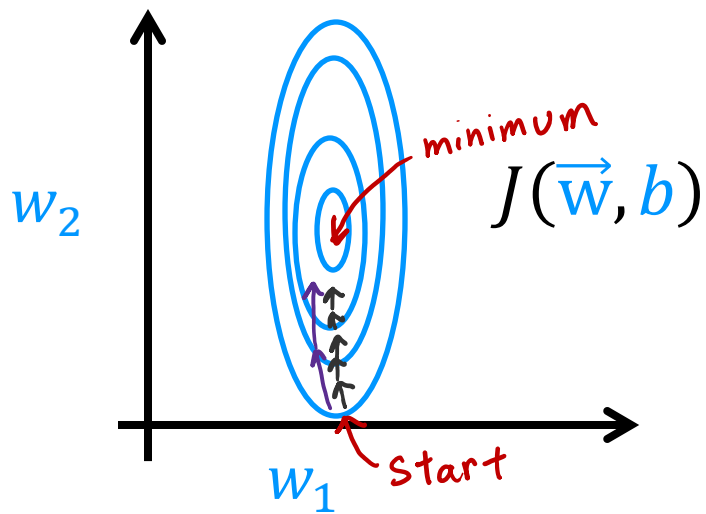
Go slower – decrease α

Adam Algorithm Intuition

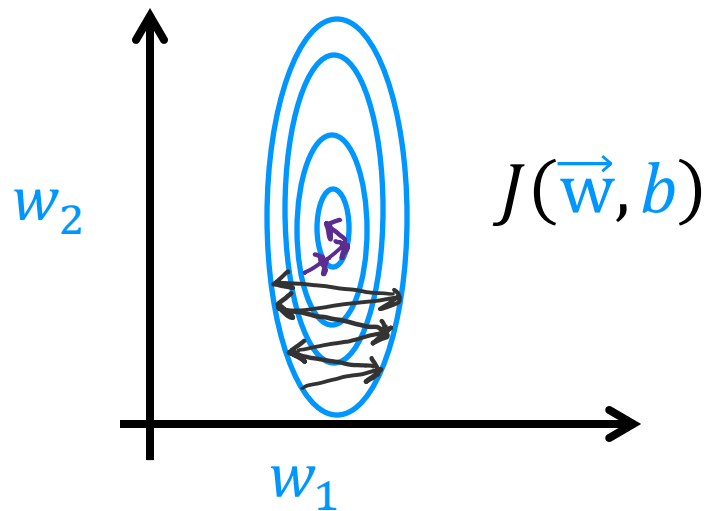
Adam: Adaptive Moment estimation *not just one α*

$$\begin{aligned}w_1 &= w_1 - \underbrace{\alpha_1}_{\text{adaptive}} \frac{\partial}{\partial w_1} J(\vec{w}, b) \\&\vdots \\w_{10} &= w_{10} - \underbrace{\alpha_{10}}_{\text{adaptive}} \frac{\partial}{\partial w_{10}} J(\vec{w}, b) \\b &= b - \underbrace{\alpha_{11}}_{\text{adaptive}} \frac{\partial}{\partial b} J(\vec{w}, b)\end{aligned}$$

Adam Algorithm Intuition



If w_j (or b) keeps moving in same direction, increase α_j .



If w_j (or b) keeps oscillating, reduce α_j .

MNIST Adam

model

```
model = Sequential([  
    tf.keras.layers.Dense(units=25, activation='sigmoid')  
    tf.keras.layers.Dense(units=15, activation='sigmoid')  
    tf.keras.layers.Dense(units=10, activation='linear')  
])
```

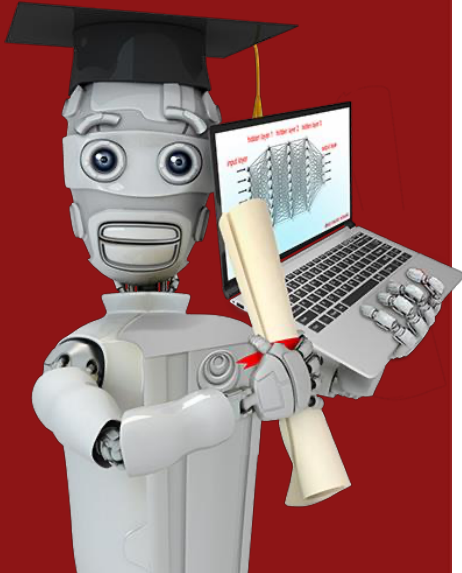
compile

$$\alpha = 10^{-3} = 0.001$$

```
model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),  
              loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True))
```

fit

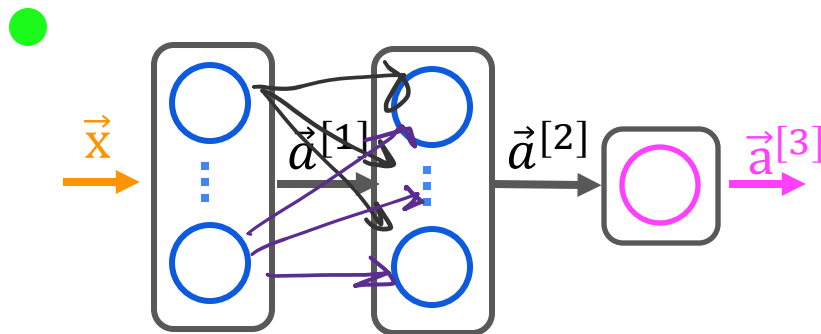
```
model.fit(X, Y, epochs=100)
```



Additional Neural Network Concepts

Additional Layer Types

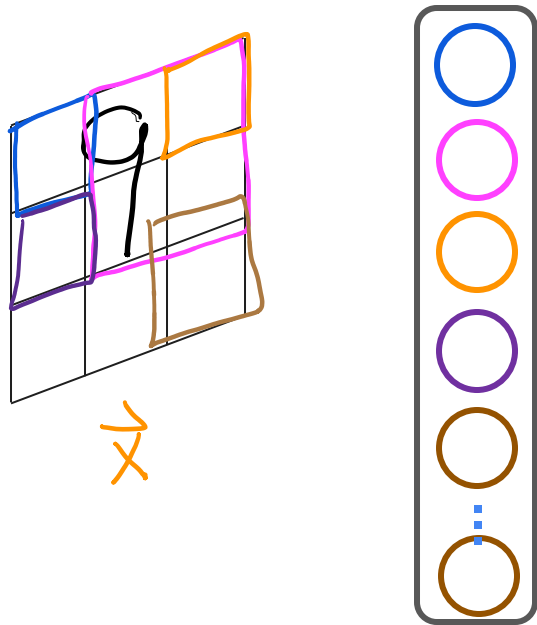
Dense Layer



Each neuron output is a function of all the activation outputs of the previous layer.

$$\vec{a}_1^{[2]} = g\left(\vec{w}_1^{[2]} \cdot \vec{a}^{[1]} + b_1^{[2]}\right)$$

Convolutional Layer

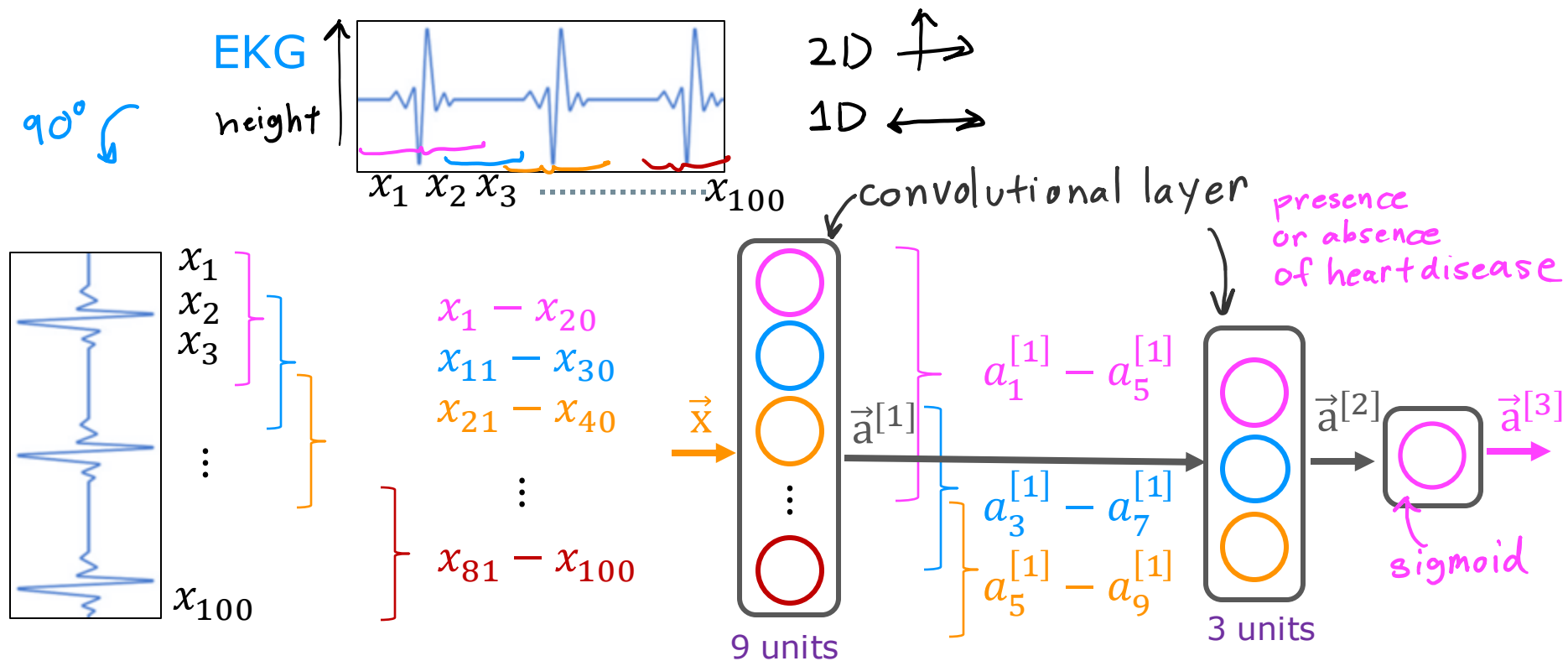


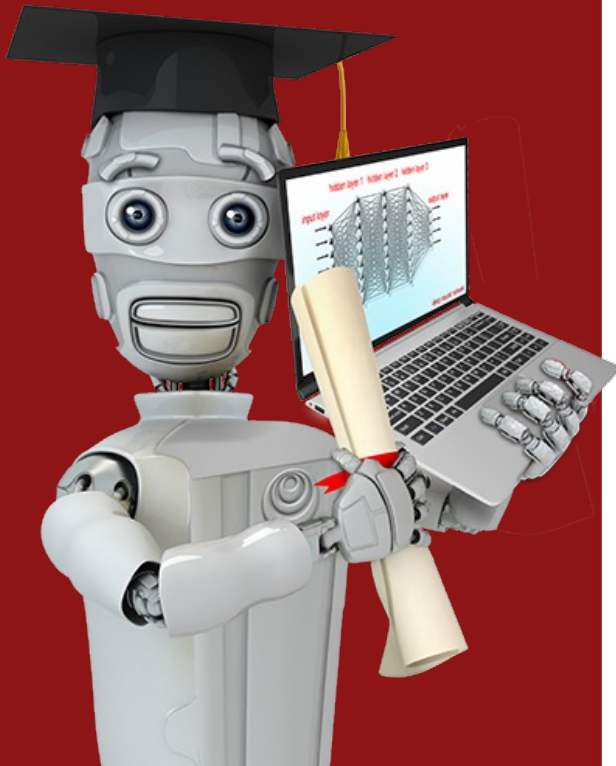
Each Neuron only looks at part of the previous layer's inputs.

Why?

- Faster computation
- Need less training data (less prone to overfitting)

Convolutional Neural Network





Backprop Intuition (Optional)

What is a derivative?

Derivative Example

Cost function $J(w) = w^2$

Say $w = 3$ $J(w) = 3^2 = 9$

If we increase w by a tiny amount $\epsilon = 0.001$ how does $J(w)$ change?

$$w = 3 + \cancel{0.001} \quad 0.002$$

$$J(w) = w^2 = \cancel{9.006001} \\ \underbrace{9.012004}_{9.012}$$

$$\epsilon = 0.002$$

$$\text{If } w \uparrow \cancel{0.001} \quad 0.002$$

$$J(w) \uparrow \quad 6 \times \cancel{0.001} \quad 0.002$$

$$\frac{\partial}{\partial w} J(w) = 6$$

$$\epsilon \leftarrow$$

$$6 \times \epsilon$$

$$6 \times 0.002 = 0.012$$

Informal Definition of Derivative

If $w \uparrow \varepsilon$ causes $J(w) \uparrow k \times \varepsilon$ then

$$\frac{\partial}{\partial w} J(w) = k$$

Gradient descent

repeat {

$$w_j = w_j - \alpha \frac{\partial}{\partial w_j} J(\vec{w}, b)$$

}

learning rate

If derivative is small, then this update step will make a small update to w_j

If the derivative is large, then this update step will make a large update to w_j

More Derivative Examples

$$w = 3$$

$$J(w) = w^2 = 9$$

$$w \uparrow 0.001$$

$$J(w) = J(3.001) = 9.006001$$

$$\frac{\partial}{\partial w} J(w) = 6$$

$$J(w) \uparrow 6 \times 0.001$$

$$w = 2$$

$$J(w) = w^2 = 4$$

$$w \uparrow 0.001$$

$$J(w) = J(2.001) = 4.004001$$

$$\frac{\partial}{\partial w} J(w) = 4$$

$$J(w) \uparrow 4 \times 0.001$$

$$w = -3$$

$$J(w) = w^2 = 9$$

$$w \uparrow 0.001$$

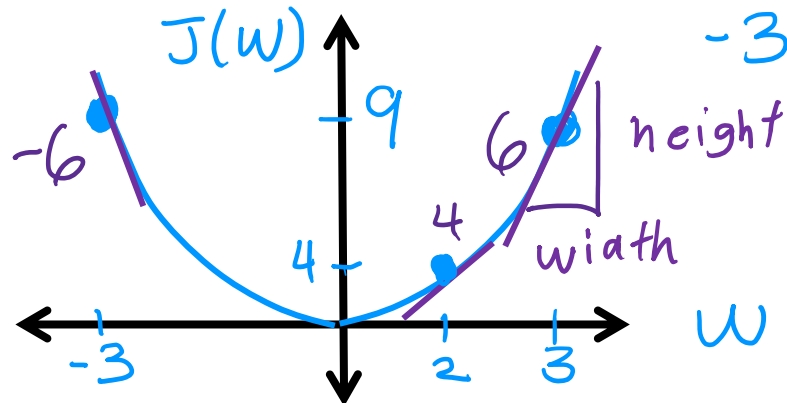
$$J(w) = J(-2.999) = 8.994001$$

$$\frac{\partial}{\partial w} J(w) = -6$$

$$J(w) \downarrow 6 \times 0.001$$

$$J(w) \uparrow -6 \times 0.001$$

$\downarrow 0.006$



Calculus	w	$\partial J(w) / \partial w$
$\frac{\partial}{\partial w} J(w) = 2w$	3	$2 \times 3 = 6$
	2	$2 \times 2 = 4$
	-3	$2 \times -3 = -6$

Even More Derivative Examples

$w = 2$	$J(w) = w^2 = 4$	$\frac{\partial}{\partial w} J(w) = 2w = 4$	$w \uparrow \underbrace{0.001}_{\varepsilon}$	$J(w) = 4.004001$ $J(w) \uparrow 4 \times \varepsilon$
	$J(w) = w^3 = 8$	$\frac{\partial}{\partial w} J(w) = 3w^2 = 12$	$w \uparrow \varepsilon$	$J(w) = 8.012006$ $J(w) \uparrow 12 \times \varepsilon$
	$J(w) = w = 2$	$\frac{\partial}{\partial w} J(w) = 1$	$w \uparrow \varepsilon$	$J(w) = 2.001$ $J(w) \uparrow 1 \times \varepsilon$
	$J(w) = \frac{1}{w} = \frac{1}{2} = 0.5$	$\frac{\partial}{\partial w} J(w) = -\frac{1}{w^2} = -\frac{1}{4}$	$w \uparrow \varepsilon$ $w = \frac{1}{2.001}$	-0.25×0.001 $0.5 - 0.00025$ $J(w) = 0.49975$ $J(w) \uparrow -\frac{1}{4} \times \varepsilon$
$\frac{\partial}{\partial w} J(w)$ $w \uparrow \varepsilon$ $J(w) \uparrow k \times \varepsilon$				

A note on derivative notation

If $J(w)$ is a function of one variable (w),

$$d \quad \frac{d}{dw} J(w)$$

If $J(w_1, w_2, \dots, w_n)$ is a function of more than one variable,

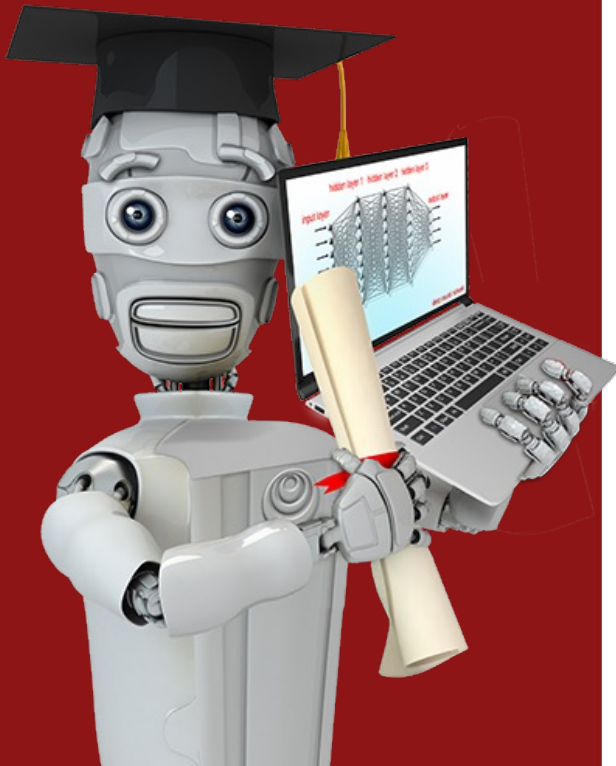
$$\bullet \quad \partial \quad \bullet \quad \frac{\partial}{\partial w_i} J(w_1, w_2, \dots, w_n) \quad \frac{\partial J}{\partial w_i} \quad \text{or} \quad \frac{\partial}{\partial w_i} J$$

"partial derivative"

notation used
in these courses



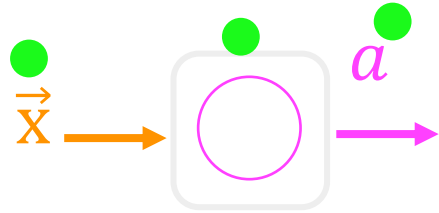
Stanford
ONLINE



Backprop Intuition (Optional)

Computation Graph

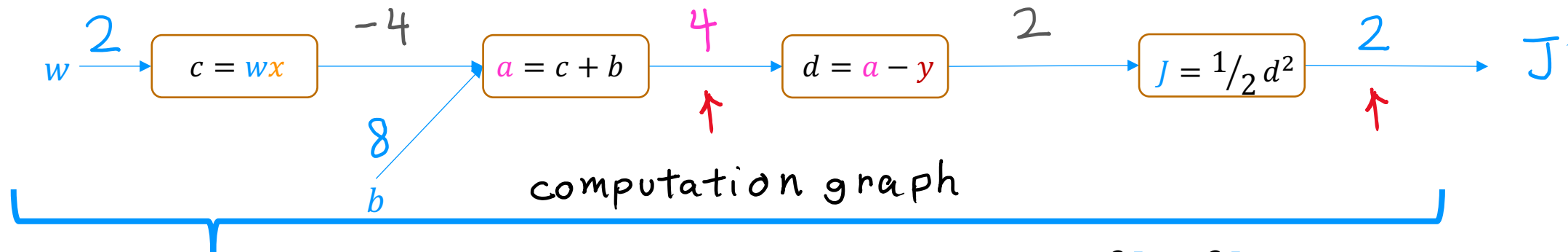
Small Neural Network Example



$$w=2 \quad b=8 \quad x=-2 \quad y=2$$

$$a = wx + b \quad \text{linear activation} \quad a = g(z) = z$$

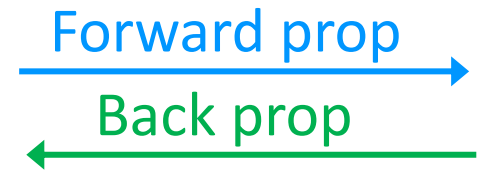
$$J(w, b) = \frac{1}{2} (a - y)^2$$



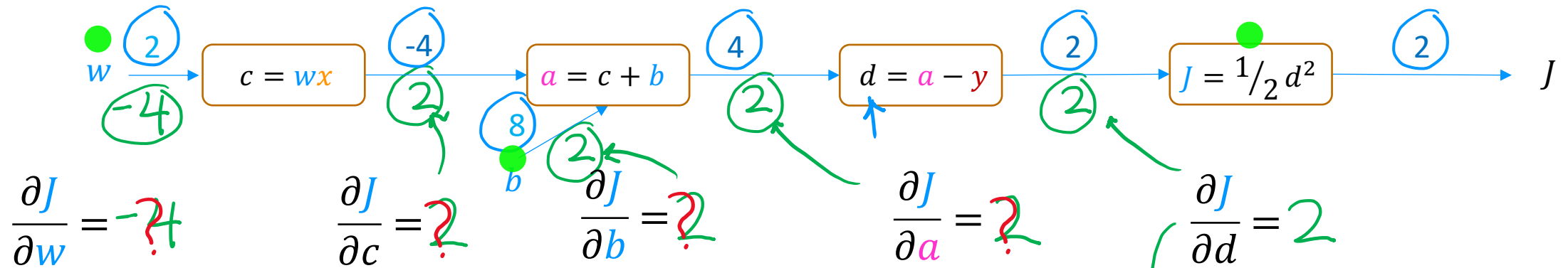
forward prop

how do we find the derivatives of J ? $\frac{\partial J}{\partial w}$ $\frac{\partial J}{\partial b}$

Computing the Derivatives



$w = 2$ $b = 8$ $x = -2$ $y = 2$ $a = wx + b$ $J = \frac{1}{2}(a - y)^2$



$w = 2.001$ $c = -4.002$

$w \uparrow 0.001$ $c \downarrow 2 \times 0.001$
 $c \uparrow -2 \times 0.001$

$J \uparrow -4 \times 0.001$

$\frac{\partial J}{\partial w} = \frac{\partial c}{\partial w} \times \frac{\partial J}{\partial c}$
 $-2 \quad 2$

$c \uparrow 0.001$ $a \uparrow 0.001$ $J \uparrow 0.002$

$\frac{\partial J}{\partial c} = \frac{\partial a}{\partial c} \times \frac{\partial J}{\partial a}$
 $1 \quad 2$

$b \uparrow 0.001$ $a \uparrow 0.001$ $J \uparrow 0.002$

$\frac{\partial J}{\partial b} = \frac{\partial a}{\partial b} \times \frac{\partial J}{\partial a}$
 $1 \quad 2$

$a \uparrow 0.001$ $d \uparrow 0.001$

$a = 4.001$ $d = 2.001$

$J \uparrow 0.002$

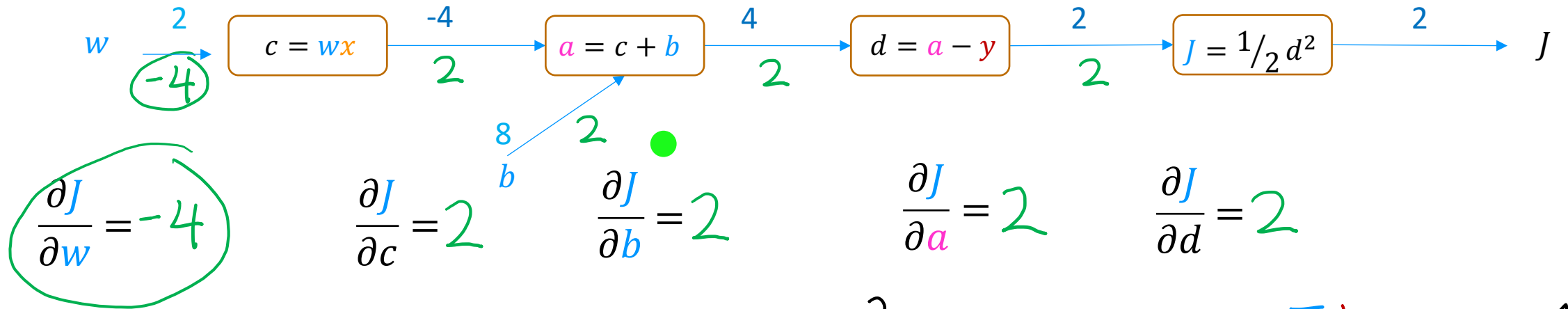
$\frac{\partial J}{\partial a} = \frac{\partial d}{\partial a} \times \frac{\partial J}{\partial d}$
 $1 \quad 2$

$d \uparrow 0.001$ $J \uparrow 0.002$
 $\epsilon \quad 2\epsilon$

chain rule
(calculus)

Computing the Derivatives

$$w = 2 \quad b = 8 \quad x = -2 \quad y = 2 \quad a = wx + b \quad J = \frac{1}{2}(a - y)^2$$



$$J = \frac{1}{2} \left((wx + b) - y \right)^2 = \frac{1}{2} \left((2 \times -2 + 8) - 2 \right)^2 = 2$$

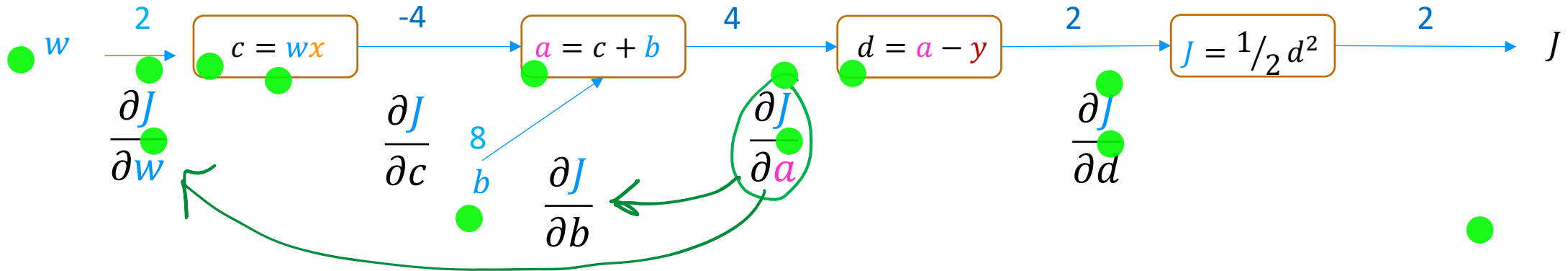
$w \uparrow 0.001$

$$J = \frac{1}{2} \left((2.001 \times -2 + 8) - 2 \right)^2 = 1.996002$$

$$\begin{aligned} J &\downarrow 4 \times 0.001 \\ J &\uparrow -4 \times 0.001 \end{aligned}$$

$$\frac{\partial J}{\partial w} = -4$$

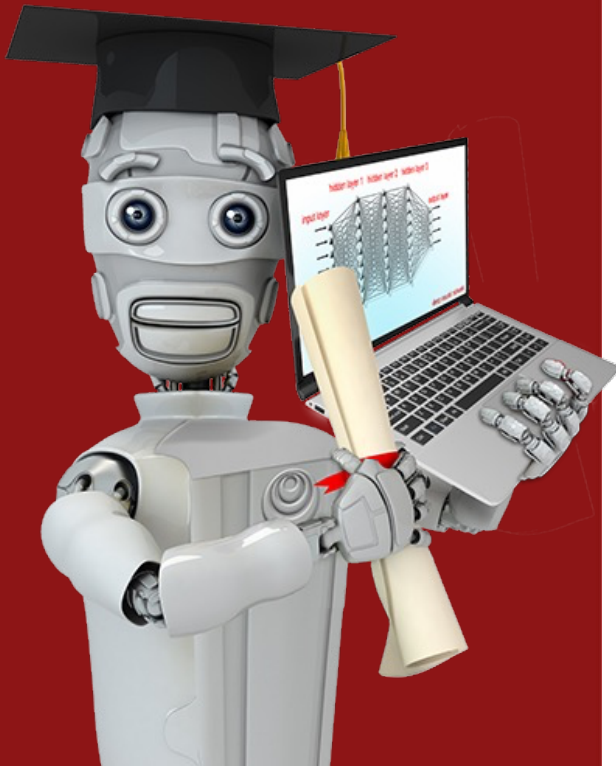
Backprop is an efficient way to compute derivatives



Compute $\frac{\partial J}{\partial a}$ once and use it to compute both $\frac{\partial J}{\partial w}$ and $\frac{\partial J}{\partial b}$.

If N nodes and P parameters, compute derivatives in roughly $N + P$ steps rather than $N \times P$ steps.

N	P	$N + P$	$N \times P$
10,000	100,000	1.1×10^5	10^9



Backprop Intuition (Optional)

Larger Neural Network Example

Neural Network Example

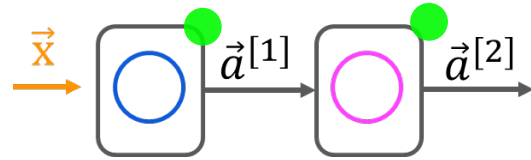
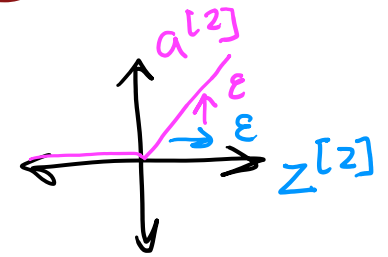
$$x = 1 \quad y = 5$$

$$w^{[1]} = 2, b^{[1]} = 0$$

$$w^{[2]} = 3, b^{[2]} = 1$$

ReLU activation

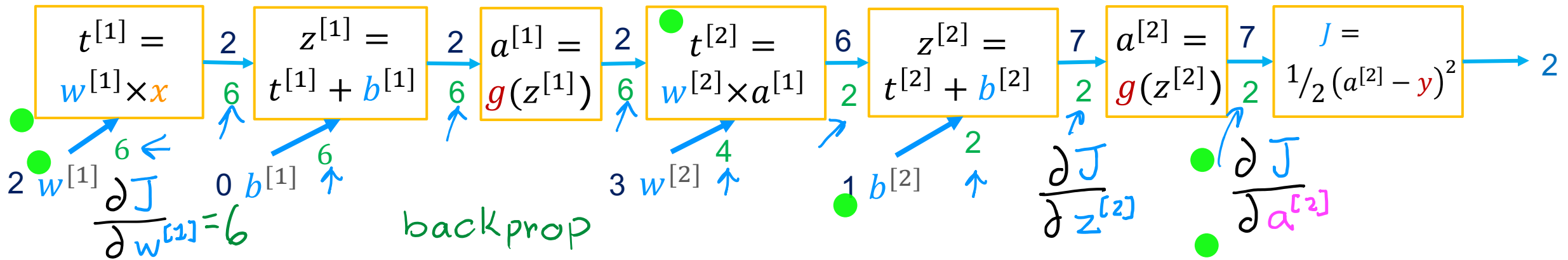
$$g(z) = \max(0, z)$$



$$a^{[1]} = g(w^{[1]}x + b^{[1]}) = \overbrace{w^{[1]}x + b^{[1]}}^{z^{[1]}} = 2 \times 1 + 0 = 2$$

$$a^{[2]} = g(w^{[2]}a^{[1]} + b^{[2]}) = \overbrace{w^{[2]}a^{[1]} + b^{[2]}}^{z^{[2]}} = 3 \times 2 + 1 = 7$$

$$J(w, b) = \frac{1}{2} (a^{[2]} - y)^2 = \frac{1}{2} (7 - 5)^2 = 2$$



if $w^{[1]} \uparrow \epsilon$, then $J \uparrow 6 \times \epsilon$ let's verify this!

Neural Network Example

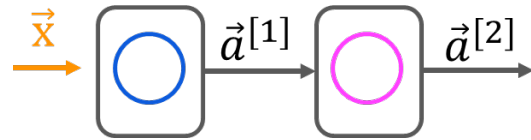
$$x = 1 \quad y = 5$$

$$w^{[1]} = 2, b^{[1]} = 0$$

ReLU activation

$$w^{[2]} = 3, b^{[2]} = 1$$

$$g(z) = \max(0, z)$$



$$a^{[1]} = g(w^{[1]}x + b^{[1]}) = w^{[1]}x + b^{[1]} = 2 \times 1 + 0 = 2$$

$$a^{[2]} = g(w^{[2]}a^{[1]} + b^{[2]}) = w^{[2]}a^{[1]} + b^{[2]} = 3 \times 2 + 1 = 7$$

$$J(w, b) = \frac{1}{2}(a^{[2]} - y)^2 = \frac{1}{2}(7 - 5)^2 = 2$$

Handwritten notes: 2.001 , 2.001 , $w^{[1]} \uparrow 0.001$, $J \uparrow 6 \times 0.001$, $\frac{\partial J}{\partial w^{[1]}} = 6$, 2.003 , 2.006 , 2.003

$$\frac{\partial J}{\partial w^{[1]}} \quad \frac{\partial J}{\partial b^{[1]}}$$

$$\frac{\partial J}{\partial w^{[2]}} \quad \frac{\partial J}{\partial b^{[2]}}$$

N nodes $\square \rightarrow \square \rightarrow \square$

P parameters
 $w_1, b_1, w_2, b_2 \dots$

inefficient way

$N \times P$

efficient way (backprop)

$N + P$